

LLM Inference Stack: LLMOps Maturity Review

Oct 8, 2026 · @Mayukh Bit

Overall verdict

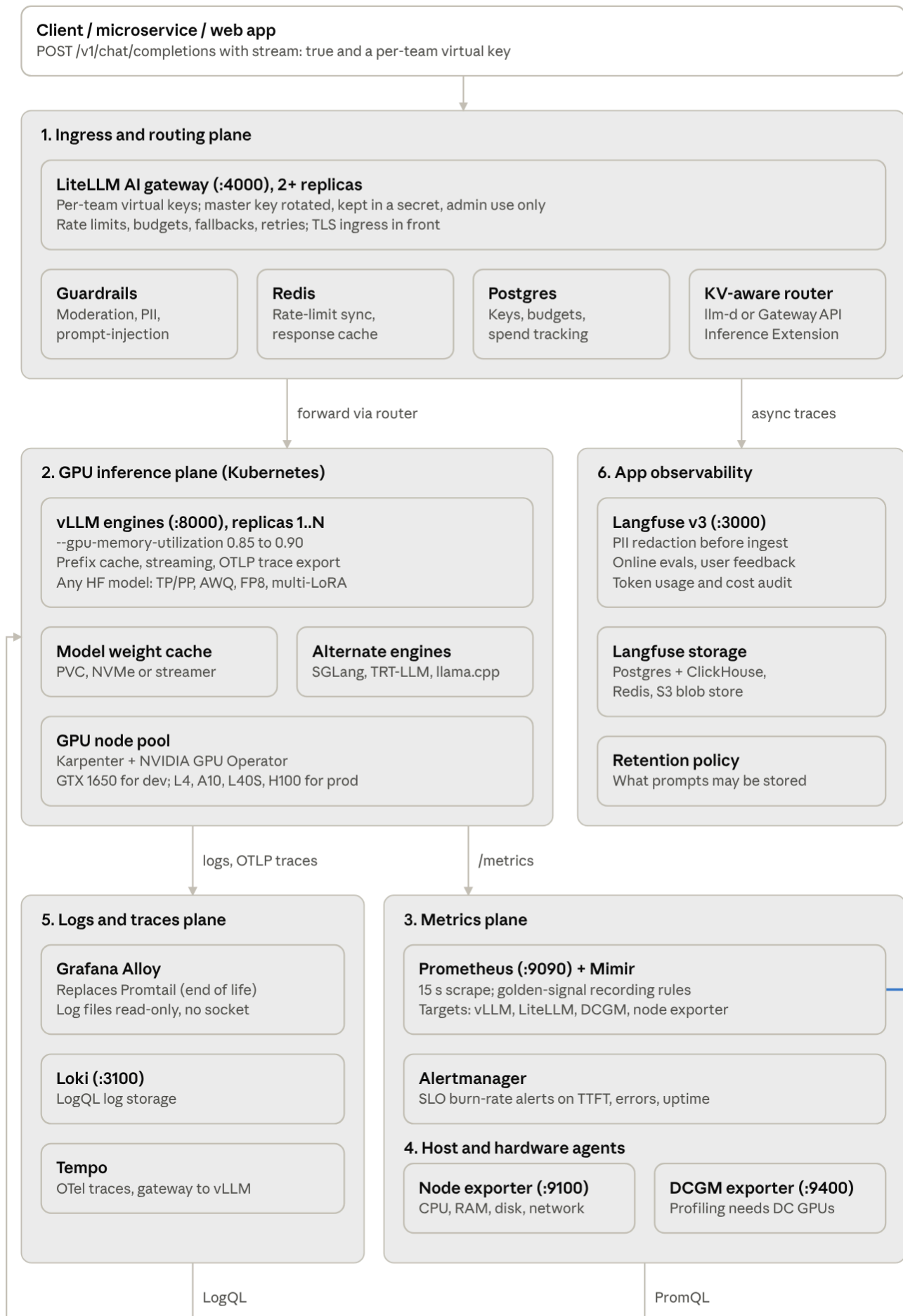
The stack is a strong observability-first proof of concept, at about **level 2 of 5** on a typical LLMOps maturity scale.

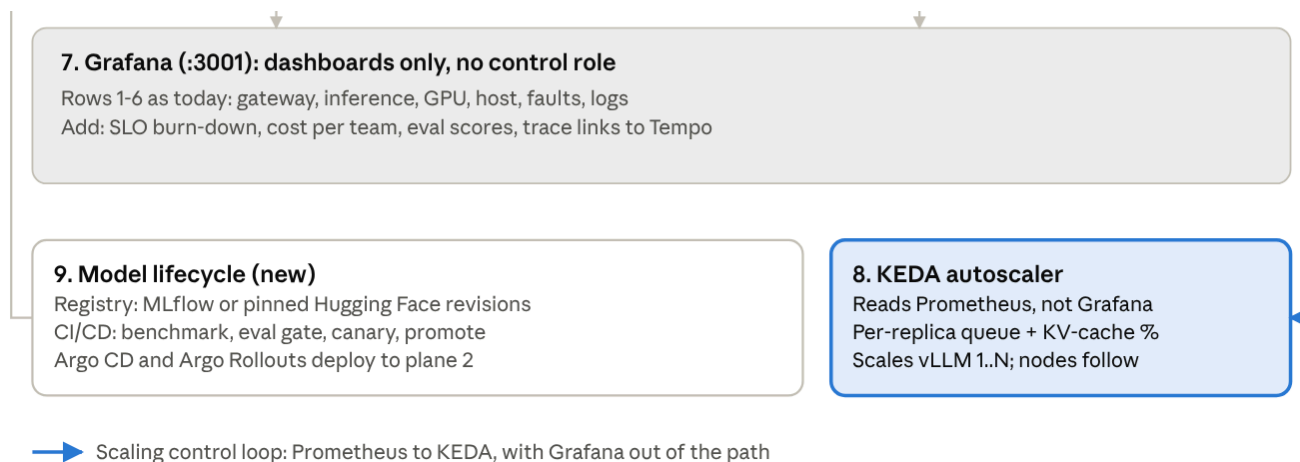
The "observe" pillar is close to production grade: gateway, golden signals, GPU telemetry, logs and LLM traces are all present. The "deploy, scale, govern, evaluate" pillars are thin or missing. The stack is also sized for a single 4 GB consumer GPU, so several components, the autoscaler especially, cannot work as drawn.

Target architecture

The updated design keeps your eight planes and their numbering, builds every fix into them, and adds plane 9 for the model lifecycle.

Your eight planes, hardened: Prometheus drives scaling, Grafana only displays





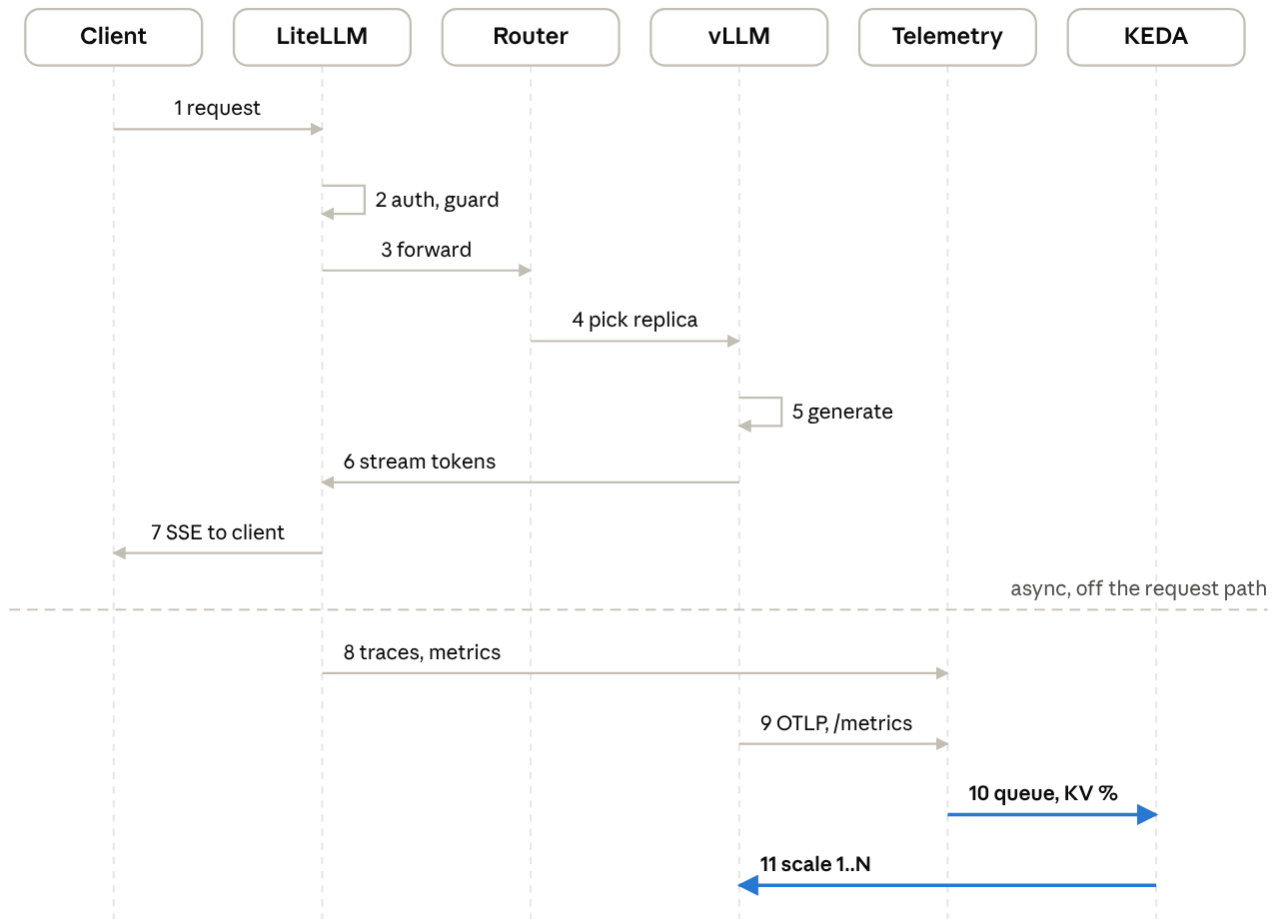
target architecture · gateway, routing, serving, platform, observability

Requests flow top-down through planes 1 and 2. Telemetry fans out to planes 3 to 6, Grafana only displays it, and the highlighted loop shows Prometheus feeding KEDA directly.

Request flow

The same 11-step sequence as today, corrected: tokens stream to the client, every hop is traced, and scaling no longer passes through Grafana.

Tokens stream to the client; telemetry feeds scaling, not Grafana



request flow · 7 synchronous steps, 4 async

1. The client posts with `stream: true` and its own virtual key, never the master key.
2. LiteLLM checks the key, rate limit and budget, then runs input guardrails.
3. LiteLLM forwards to the KV-aware router instead of a fixed `vllm:8000` host.
4. The router picks the replica most likely to hold the prompt's prefix in KV cache.
5. vLLM schedules continuous batches and generates tokens.
6. Tokens stream back to LiteLLM, which runs output guardrails.
7. LiteLLM streams tokens to the client as server-sent events, so the client sees real time to first token.
8. LiteLLM sends redacted traces to Langfuse and serves RPS and latency to Prometheus; the trace ID travels with the request.
9. vLLM exports OTLP spans to Tempo and serves TTFT, tokens/s, queue and KV-cache metrics to Prometheus; Alloy ships its logs to Loki.
10. KEDA queries Prometheus for per-replica queue depth and KV-cache use.
11. KEDA scales vLLM replicas, and Karpenter adds GPU nodes when pods can't be placed.

Grafana reads Loki, Prometheus and Tempo for dashboards and plays no part in this flow.

Fix: things that are wrong or won't work as drawn

Six issues in the current design need correcting before anything is added.

1. **The autoscaler should not hang off Grafana.** Grafana is a viewer, and nothing in the control path should depend on it. KEDA should read Prometheus directly through its Prometheus scaler.
2. **Scaling on queue depth > 5 alone is too crude.**
 - Normalize per replica, e.g. `sum(vllm:num_requests_waiting) / count(replicas)`.
 - Combine it with KV-cache utilization (the vLLM KV-cache usage gauge; the name varies by version). KV cache is the real saturation signal for LLMs.
 - Add a scale-down stabilization window. Model loading takes seconds to minutes, so flapping is expensive.
3. **Scaling 1 to N pods on one GTX 1650 is impossible.** Pod autoscaling only helps when there are GPUs to land on. Real scaling needs:
 - a GPU node pool
 - a node autoscaler (Karpenter or Cluster Autoscaler)
 - the NVIDIA GPU Operator
 - a model-weight cache (PVC, local NVMe or a model streamer) to keep cold starts short
4. **Swap Promtail for Grafana Alloy or an OpenTelemetry Collector.** Promtail is deprecated in favor of Alloy and has reached end of life. Alloy can also ship metrics and traces, so it can be the single collection agent.
5. **Check the Langfuse version.** Langfuse v3 needs ClickHouse, Redis and S3-compatible blob storage, not just Postgres. On v2, plan the migration.
6. **Expect some empty GPU panels on a GeForce card.** DCGM profiling metrics (memory bandwidth, SM occupancy) are largely unsupported on consumer GPUs. The Bandwidth panel will likely stay empty until datacenter GPUs (T4, L4, A10 and up). Utilization, memory, temperature and power will work.

Findings from the detailed design

The component and sequence diagrams show one urgent security issue: clients call the gateway with the LiteLLM master key. Fix that before anything else.

Finding	Risk	Fix
Clients send <code>Bearer sk-litellm-master-key-...</code>	Master key grants full admin; it appears in the diagram, so treat it as leaked	Rotate it now; issue per-team virtual keys; keep the master key in a secret, admin use only
Docker Compose hostnames (<code>vllm:8000</code>) and the Docker socket, but KEDA in the design	KEDA runs only on Kubernetes, so the scaling box can't work on Compose	Move to Kubernetes, or mark autoscaling as future work until then
Promtail mounts <code>/var/run/docker.sock</code>	The socket is root-equivalent access to the host	Read container log files read-only, or put a read-only socket proxy in front
vLLM <code>--gpu-memory-utilization 0.60</code> (~1,952 MiB of 4 GB)	About 2 GB of VRAM sits idle, capping KV cache and concurrency	On a dedicated GPU, raise to 0.85 to 0.90 and load-test
Prometheus scrape interval 5 s	Fine on one box; costly with many targets	Use 15 s by default; KEDA polls Prometheus on its own interval
Langfuse on Postgres only (:3000)	That is the v2 layout; v3 needs ClickHouse, Redis and S3	Plan the v3 migration
Full prompts and completions sent to Langfuse	PII lands in trace storage	Redact or mask before ingest; set a retention policy
vLLM streams to LiteLLM, which returns plain JSON to the client (steps 4 to 5)	Clients never see real time to first token	Pass <code>stream: true</code> end to end; record TTFT at the gateway too
No trace context between LiteLLM and vLLM	Gateway and engine latency can't be joined per request	Propagate OTel <code>traceparent</code> and enable vLLM's OTLP export
Grafana's queue-backlog alert drives KEDA	Confirms the control-loop issue above	Point the KEDA ScaledObject at Prometheus directly

Add: missing pieces for end-to-end LLMOps

The largest gaps are KV-aware routing, alerting, model lifecycle and evaluation.

Serving and routing

- **A KV-cache-aware router** between LiteLLM and the vLLM replicas. Plain Kubernetes round-robin wastes prefix-cache hits. Options: llm-d, Gateway API Inference Extension, or the vLLM production-stack router.
- **A serving control plane** (KServe, llm-d or the vLLM production stack) to manage deployments, revisions and canaries declaratively.
- **Multi-GPU and multi-node support** for large models: tensor and pipeline parallelism, plus LeaderWorkerSet or Ray.
- **Quantization paths** (AWQ, GPTQ, FP8) and **multi-LoRA serving**.
- **Prefix caching** in vLLM, and a semantic or exact response cache in LiteLLM backed by Redis.

Gateway hardening

- **LiteLLM's own Postgres** for virtual keys, per-team budgets and spend tracking.
- **Redis** to sync rate limits across LiteLLM replicas; run at least 2 replicas.
- **Fallbacks and retries** across models and providers, with per-model timeouts.
- **Guardrails** through LiteLLM's guardrail hooks: input and output moderation (e.g. Llama Guard), PII redaction (e.g. Presidio), prompt-injection checks.
- TLS ingress, secrets management (Vault or External Secrets) and network policies.

Reliability and operations

- **Alertmanager** with SLO-based burn-rate alerts on TTFT, error rate and availability. Dashboards without alerts aren't ops.
- **Prometheus recording rules** for golden signals, and long-term storage (Mimir or Thanos).
- **End-to-end OpenTelemetry tracing** from gateway to vLLM (vLLM exports OTLP), stored in Tempo, to follow one slow request across layers.
- **Load testing and benchmarking** in CI (guidellm or vLLM's benchmark tools) before every model or config change, to set capacity per GPU type.

Model lifecycle

- **A model registry** with versioning (MLflow, or Hugging Face with pinned revisions).

- **CI/CD for models:** build, benchmark, eval gate, canary or shadow rollout, then promote or roll back. Argo CD or Argo Rollouts fits.
- **Evaluation:**
 - offline suites before deploy (lm-eval-harness or custom golden sets)
 - online evals in Langfuse (LLM-as-judge, user feedback scores)
 - drift and quality dashboards beside the latency ones

Governance and cost

- Cost per request, team and model: LiteLLM spend tracking plus GPU-hour attribution.
- Audit logging and a data-retention policy for prompts and completions, including what Langfuse may store.

Replace or reconsider

Five swaps tighten the design; the only true removal is Grafana as a control-plane dependency.

Current	Suggestion	Why
Promtail	Grafana Alloy or OTel Collector	Promtail is end of life; one agent for all telemetry
Grafana to KEDA trigger	Prometheus to KEDA	Keeps the control loop independent of the UI
Queue-depth-only HPA	Per-replica queue + KV-cache % (optionally TTFT)	Reflects real LLM saturation
vLLM-only engine	vLLM by default, engine pluggable behind LiteLLM	SGLang, TensorRT-LLM or llama.cpp fit some workloads better
GTX 1650 dev box	Keep for dev; L4, A10, L40S or H100 tiers for prod	FlashAttention, FP8 and DCGM profiling need newer GPUs

Host metrics (Row 4) and Loki are worth keeping.

Supporting most kinds of LLMs

vLLM plus LiteLLM is the right default: it covers most Hugging Face decoder models, vision-language models, embeddings and rerankers. Reaching "most models" needs four changes.

- **Large models:** tensor and pipeline parallelism, plus multi-node serving.
- **Engine heterogeneity:** llama.cpp for edge or CPU and TensorRT-LLM for max-throughput NVIDIA, all behind the same LiteLLM endpoint.
- **Per-model deployment templates** (GPU type, max context, quantization, parallelism), so adding a model is config, not code.
- **Disaggregated prefill and decode** at high volume (llm-d supports this).

Suggested priority order

Start with the control-loop fix and alerting, then harden, then scale.

- ☐ Rewire KEDA to read Prometheus directly
- ☐ Add Alertmanager and SLO burn-rate alerts
- ☐ Add Postgres and Redis for LiteLLM, plus guardrails
- ☐ Swap in Alloy and add OpenTelemetry tracing
- ☐ Add a load-test and eval gate in CI
- ☐ Adopt a serving control plane with KV-aware routing
- ☐ Set up a GPU node pool with node autoscaling and weight caching